UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE CIENCIAS

Departamento Académico de Ciencias de la Computación

CC3S2 — Desarrollo de Software (Ciclo 2026-1)

Documento de Diseño de Software (SDD)

Sistema Geoespacial de Logística Inversa y Recolección de Residuos Sólidos (**EcoSystem**)

Especificación de Arquitectura, Modelado de Dominio y Patrones de Diseño

Brian Jair La Torre Urbina
Angel Andres Alvinagorta Huapaya
Omar Ccahuana Sosa
Jhiens Angel Guerrero Ccompi
Emhir Carlo Andre Rodriguez Ricra

Docente: Prof. Manuel Quispe Torres

20 de octubre de 2026

Resumen

El presente Documento de Diseño de Software (*Software Design Description* — SDD), elaborado bajo los lineamientos del estándar IEEE 1016-2009, especifica la arquitectura de software, la descomposición modular y el diseño detallado orientado a objetos del sistema **EcoSystem**. A partir de los requisitos funcionales y no funcionales definidos en la Especificación de Requisitos de Software (norma ISO/IEC/IEEE 29148), el sistema se estructura como un monolito modular bajo una arquitectura cerrada de cuatro capas, particionada funcionalmente en subdominios de negocio. Se detallan los diagramas estructurales UML, los modelos de persistencia espacial con índices GiST sobre PostGIS, los contratos de interfaz y la aplicación de los patrones de diseño GoF (Singleton, Strategy, Adapter y Observer) e interacciones GRASP que dan soporte a los subsistemas de autenticación y ruteo óptimo (TSP), gestión geoespacial de contenedores, recolección transaccional Drop & Go, gobernanza paramétrica y fiscalización ambiental georreferenciada.

Palabras clave: Arquitectura en capas cerradas, Diseño Orientado a Objetos, UML, Patrones GoF, PostGIS, Índice GiST, Problema del Viajante (TSP), IEEE 1016.

Índice

1. Introducción y Contexto Arquitectónico **3**
1.1. Propósito del Documento 3
1.2. Entorno Tecnológico 3

2. Arquitectura del Sistema: Cuatro Capas Cerradas **3**
2.1. Topología y Regla de Dependencia 3
2.2. Descripción de las Capas Arquitectónicas 3
2.3. Descomposición Modular por Subdominios (*Vertical Slicing*) 4

3. Diseño Detallado y Modelado UML por Subsistema **4**
3.1. Subsistema de Infraestructura Core, Autenticación y Motor TSP 4
3.1.1. Instanciación de Infraestructura (Patrón Singleton) 4
3.1.2. Modelo de Autenticación y Control de Acceso (RBAC) 4
3.1.3. Motor de Optimización de Rutas TSP (Patrones Strategy y Adapter) . . . 5
3.2. Subsistema Geoespacial PostGIS, Contenedores y Umbrales 5
3.2.1. Modelado Espacial Esférico e Indexación GiST 5
3.2.2. Política de Transición de Estados por Umbral 6
3.3. Subsistema Drop & Go y Notificaciones Reactivas (Patrón Observer) 6
3.3.1. Modelo Transaccional de Paquetes Reciclables 6
3.3.2. Desacoplamiento de Eventos de Vaciado mediante Patrón Observer 6
3.4. Subsistema de Gobernanza Paramétrica y Fiscalización Ambiental 7
3.4.1. Externalización de Parámetros de Negocio 7
3.4.2. Validación Georreferenciada de Denuncias Ambientales 7

4. Especificación de Interfaces REST (OpenAPI 3.1) **8**

5. Síntesis Arquitectónica y Extensibilidad **8**
5.1. Evaluación del Diseño Actual 8
5.2. Puntos de Extensión Arquitectónica 9

1. Introducción y Contexto Arquitectónico

1.1. Propósito del Documento

Este documento establece la especificación técnica del diseño de **EcoSystem**, plataforma orientada a la logística inversa de residuos reciclables y a la fiscalización ambiental en el distrito del Rímac. Su objetivo es describir la estructura estática y dinámica del software, las decisiones arquitectónicas, el modelado espacial de datos y los mecanismos de desacoplamiento entre componentes mediante diagramas UML y contratos de interfaz.

1.2. Entorno Tecnológico

El sistema sigue un diseño cliente-servidor bajo el paradigma API-First. El núcleo de servidor está implementado en Python 3.12 utilizando el framework FastAPI para la exposición de servicios RESTful, SQLAlchemy 2.0 como mapeador objeto-relacional (ORM) y GeoAlchemy2 para la manipulación de tipos espaciales. La capa de persistencia opera sobre PostgreSQL 16 con la extensión geoespacial PostGIS 3.4, mientras que el cliente web integra Leaflet.js para el renderizado cartográfico e interacción georreferenciada.

2. Arquitectura del Sistema: Cuatro Capas Cerradas

2.1. Topología y Regla de Dependencia

EcoSystem implementa una **arquitectura cerrada de cuatro capas** (Figura 1). En esta topología, las dependencias fluyen estrictamente hacia abajo entre capas adyacentes: una capa solo puede invocar componentes pertenecientes a la capa inmediatamente inferior.

Esta restricción impide que la capa de presentación realice consultas directas al motor de base de datos o invoque adaptadores de red externos, garantizando que toda regla de negocio y validación transaccional sea procesada por la capa de control.

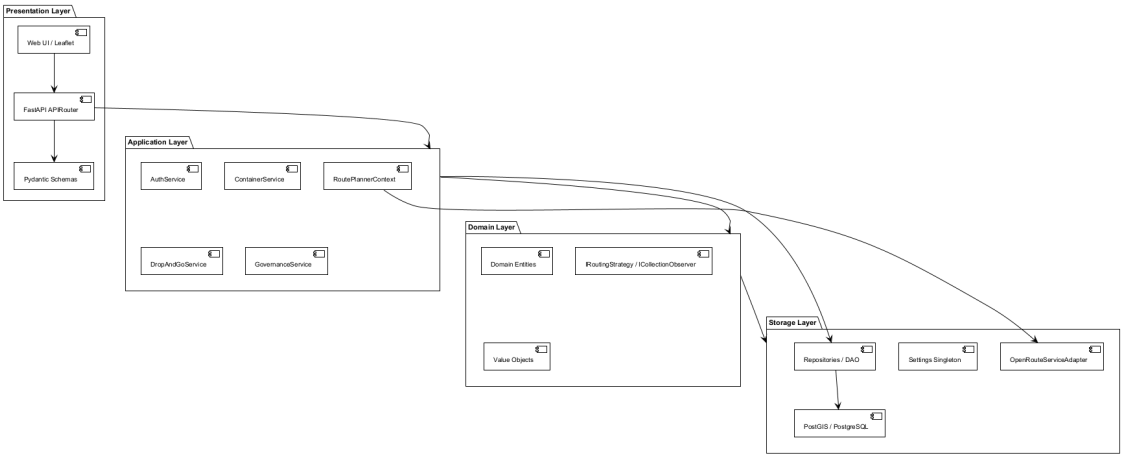


Figura 1: Diagrama de Arquitectura Cerrada de 4 Capas de EcoSystem.

2.2. Descripción de las Capas Arquitectónicas

1. **Capa de Presentación y Frontera (*Boundary / Presentation Layer*):** Integrada por los controladores HTTP (`APIRouter`) y esquemas de transferencia de datos (`DTOs`) implementados en `Pydantic V2` bajo el estándar OpenAPI 3.1. Gestiona la validación sintáctica de las cargas útiles JSON, el control de respuestas HTTP y la interfaz cartográfica en Leaflet.js.

2. **Capa de Aplicación y Control (*Application / Control Layer*):** Aplica el principio Controlador (GRASP) a través de servicios de aplicación (`AuthService`, `ContainerService`, `RoutePlannerContext`, `DropAndGoService`, `GovernanceService`, `ComplaintService`). Coordina los casos de uso, gestiona las transacciones y ejecuta las políticas del negocio de manera agnóstica a la tecnología de persistencia.

3. **Capa de Entidades de Dominio (*Domain Entities Layer*):** Representa el núcleo conceptual del sistema. Contiene las entidades persistentes (`User`, `Container`, `DropAndGoPackage`, `SystemParameter`, `WasteReport`), objetos de valor inmutables (`GeoNode`) y las interfaces polimórficas que definen contratos algorítmicos y de eventos (`IRoutingStrategy`, `IExternalDistancePort`, `ICollectionObserver`).

4. **Capa de Almacenamiento e Infraestructura (*Storage Layer*):** Encapsula el acceso a datos mediante el patrón **Repository** (`UserRepository`, `ContainerRepository`, entre otros) sobre PostgreSQL/PostGIS, así como los adaptadores concretos hacia servicios web cartográficos externos.

2.3. Descomposición Modular por Subdominios (*Vertical Slicing*)

Complementando la separación horizontal en cuatro capas, el sistema se particiona verticalmente en subdominios funcionales autónomos (*Vertical Slices*). Cada módulo encapsula sus propios esquemas de frontera, servicios, entidades y repositorios, compartiendo únicamente el núcleo transversal de configuración y conexión a datos (`app/core/`).

Tabla 1: Matriz de Descomposición Modular y Patrones Asociados

Subsistema	**Paquete Base**	**Entidades y Objetos Clave**
Core y Autenticación	`app/core/`, `app/modules/auth/`	`Settings`, `User`, `UserRole`
Ruteo Logístico TSP	`app/modules/routing/`	`GeoNode`, `RoutePlannerContext`
Gestión Geoespacial	`app/modules/containers/`	`Container`, `ContainerCitizenReport`
Recolección Drop & Go	`app/modules/drop_and_go/`	`DropAndGoPackage`, `ContainerEmptiedEvent`
Gobernanza y Fiscalización	`app/modules/governance/`, `app/modules/complaints/`	`SystemParameter`, `WasteReport`

3. Diseño Detallado y Modelado UML por Subsistema

3.1. Subsistema de Infraestructura Core, Autenticación y Motor TSP

3.1.1. Instanciación de Infraestructura (Patrón Singleton)

Para garantizar una única instancia global de la configuración de entorno y del pool de conexiones hacia PostgreSQL durante todo el ciclo de vida del proceso servidor, se aplica el patrón creacional **Singleton**. La clase `Settings` (derivada de `BaseSettings` de Pydantic) y el objeto `Engine` de SQLAlchemy se proveen mediante funciones de acceso decoradas con memorización (`@lru_cache()`) en `app/core/config.py`, evitando la sobrecarga por apertura reiterada de sockets de base de datos.

3.1.2. Modelo de Autenticación y Control de Acceso (RBAC)

El control de acceso se modela mediante la entidad `User` y la enumeración cerrada `UserRole`, la cual contempla seis perfiles: `CIUDADANO`, `CONDUCTOR`, `OPERARIO`, `ADMIN`, `COORDINADOR` y `SUPERVISOR`.

Por diseño de seguridad, el esquema de entrada del endpoint de registro público (`POST /api/v1/auth/register`) omite el atributo de rol, forzando en la capa de servicio la asignación del perfil `CIUDADANO` para impedir vulnerabilidades de escalada de privilegios. La verificación de identidad se realiza mediante hashes criptográficos de contraseña y emisión de tokens JWT sin estado (*stateless*).

3.1.3. Motor de Optimización de Rutas TSP (Patrones Strategy y Adapter)

El cálculo del circuito óptimo de recolección para los camiones municipales resuelve una instancia del Problema del Viajante (*Traveling Salesperson Problem* — TSP) sobre el conjunto de contenedores que han alcanzado el estado `LLENO`, partiendo desde el depósito municipal. El diseño de este subsistema se muestra en la Figura 2.

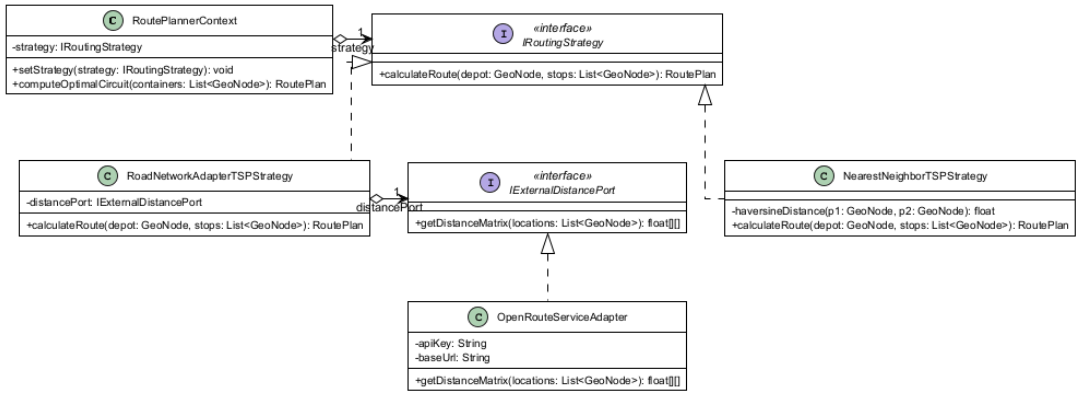


Figura 2: Diagrama de Clases UML: Patrones Strategy y Adapter en el motor de ruteo TSP.

- **Patrón Strategy (Principio Abierto/Cerrado — OCP):** La clase de contexto `RoutePlannerContext` mantiene una referencia a la interfaz abstracta `IRoutingStrategy`, la cual declara el método polimórfico `calculate_optimal_route(depot: GeoNode, targets: List[GeoNode])`. Esto permite intercambiar el algoritmo de resolución en tiempo de ejecución:
  1. `NearestNeighborTSPStrategy`: Implementa una heurística constructiva voraz (*greedy*) de complejidad temporal $O(N^2)$ basada en el cálculo geométrico directo mediante la fórmula ortodrómica de Haversine, operativa sin dependencia de red externa.
  2. `RoadNetworkAdapterTSPStrategy`: Calcula la secuencia óptima considerando la topología vial real (sentidos de calles y distancias de conducción).
- **Patrón Adapter (Principio de Inversión de Dependencias — DIP):** Para evitar que el dominio dependa directamente de las firmas JSON y protocolos HTTP de un proveedor cartográfico específico, se define el puerto de dominio `IExternalDistancePort`. La clase concreta `OpenRouteServiceAdapter` implementa dicha interfaz, encapsulando las peticiones hacia la API externa y adaptando sus matrices de distancia al formato requerido por `RoadNetworkAdapterTSPStrategy`.

3.2. Subsistema Geoespacial PostGIS, Contenedores y Umbrales

3.2.1. Modelado Espacial Esférico e Indexación GiST

La representación de los puntos de acopio se centraliza en la entidad `Container`, cuyo diseño estructural e interacción con la capa de persistencia se detallan en la Figura 3. La ubicación geográfica se define mediante el tipo nativo `Geography` de PostGIS bajo el sistema de referencia espacial WGS84 (`SRID 4326`):

```
location = Column(Geography(geometry_type='POINT', srid=4326))
```

A diferencia del tipo plano `Geometry`, el modelo `Geography` ejecuta cálculos sobre un modelo esferoidal terrestre, expresando las distancias directamente en metros. Para optimizar las búsquedas espaciales, se define un índice basado en árboles de búsqueda generalizados (**GiST**) sobre la columna `location`. El repositorio `ContainerRepository` encapsula las consultas de proximidad utilizando el predicado espacial `ST_DWithin`, logrando tiempos de ejecución inferiores a 150 ms, dentro del umbral establecido por el requisito no funcional **RNF-001** ($< 2{,}0$ **s**).

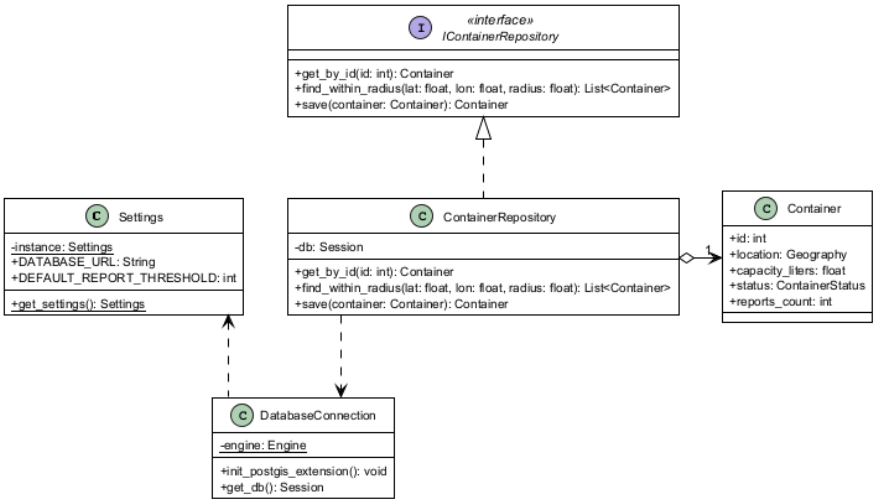


Figura 3: Diagrama de Clases UML: Patrones Singleton y Repository con integración espacial PostGIS.

3.2.2. Política de Transición de Estados por Umbral

El control de saturación de contenedores opera mediante reportes ciudadanos colaborativos modelados en la entidad `ContainerCitizenReport`. Cuando `ContainerService` recibe una notificación de llenado, verifica que el ciudadano no haya emitido un reporte previo para la misma unidad en el ciclo actual (control de idempotencia y prevención de saturación). Al incrementarse el contador de reportes válidos y alcanzar el umbral parametrizado del sistema, la entidad `Container` transiciona su estado de `DISPONIBLE` a `LLENO`, quedando habilitada automáticamente para la consulta del motor TSP.

3.3. Subsistema Drop & Go y Notificaciones Reactivas (Patrón Observer)

3.3.1. Modelo Transaccional de Paquetes Reciclables

El servicio `DropAndGoService` gestiona la entrega de bolsas de material reciclable preclasificado por los vecinos. A cada solicitud se le asigna un código único normalizado (`#ECO-2026-XXXX`) integrable en etiquetas QR. La entidad `DropAndGoPackage` mantiene los atributos de tipo de residuo, peso estimado, referencia foránea al contenedor de depósito y el estado del ciclo de vida del paquete (`PENDIENTE`, `RECOLECTADO`, `EN_PLANTA`).

3.3.2. Desacoplamiento de Eventos de Vaciado mediante Patrón Observer

El acto físico de vaciar un contenedor por parte del operario municipal (`POST /api/v1/containers/{id}/empt` requiere actualizar simultáneamente el estado del contenedor, la trazabilidad de todos los paquetes depositados en su interior y las alertas dirigidas a los vecinos de la zona.

Para evitar acoplar estas tres responsabilidades dentro de un único método monolítico, se diseñó una arquitectura orientada a eventos internos mediante el patrón de comportamiento **Observer** (Figura 4):

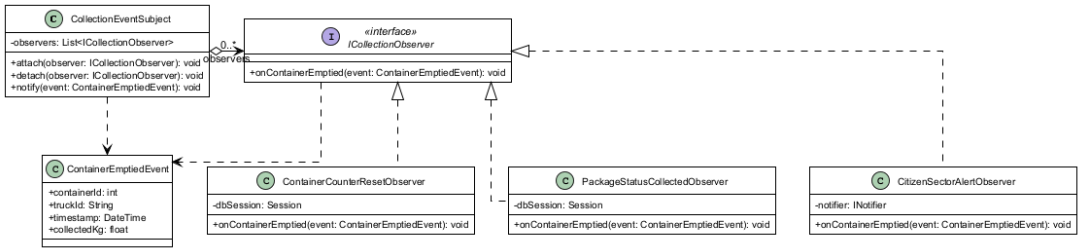


Figura 4: Diagrama de Clases UML: Patrón Observer en eventos de recolección y vaciado.

- **Sujeto Publicador (`CollectionEventSubject`):** Administra la lista de suscriptores acoplados únicamente a la interfaz `ICollectionObserver` y propaga el objeto de evento inmutable `ContainerEmptiedEvent`.
- `ContainerCounterResetObserver`**:** Suscriptor encargado de reiniciar a cero el contador de reportes ciudadanos del contenedor vaciado y restituir su estado a `DISPONIBLE`.
- `PackageStatusCollectedObserver`**:** Suscriptor que consulta los registros `DropAndGoPackage` asociados al contenedor y transiciona su estado de `PENDIENTE` a `RECOLECTADO`.
- `CitizenSectorAlertObserver`**:** Suscriptor responsable de emitir la notificación de disponibilidad del punto limpio hacia los usuarios ubicados en el radio de influencia del contenedor.

Este diseño satisface el principio de **Bajo Acoplamiento (GRASP)** y el principio **Open/Closed (OCP)**, permitiendo incorporar nuevos efectos secundarios en el futuro (como auditorías o recálculo de métricas) añadiendo nuevos observadores sin alterar el sujeto publicador.

3.4. Subsistema de Gobernanza Paramétrica y Fiscalización Ambiental

3.4.1. Externalización de Parámetros de Negocio

Para evitar que las reglas cuantitativas de operación queden codificadas de forma rígida (*hardcoded*) en la capa de control, el módulo de gobernanza introduce la entidad persistente `SystemParameter` sobre la tabla `system_parameters`.

El servicio `GovernanceService` expone operaciones protegidas bajo el rol `ADMIN` que permiten consultar y actualizar en tiempo de ejecución parámetros críticos del dominio:

- `DEFAULT_CONTAINER_REPORT_THRESHOLD`: Entero que determina el número de reportes necesarios para declarar un contenedor como `LLENO`.
- Factores de bonificación por kilogramo de material reciclado (`PLASTICO`, `VIDRIO`, `METAL`).

3.4.2. Validación Georreferenciada de Denuncias Ambientales

El módulo de fiscalización gestiona el reporte de puntos de arrojo clandestino de residuos mediante la entidad `WasteReport`, la cual vincula la evidencia fotográfica (recibida como flujo `multipart/form-data`) con las coordenadas GPS de captura y la marca temporal.

En la capa de aplicación, `ComplaintService` ejecuta una regla de validación espacial previa a la persistencia: verifica que la latitud y longitud suministradas se encuentren contenidas dentro del polígono delimitador (*bounding box*) correspondiente a los límites jurisdiccionales del distrito del Rímac (latitud $\in [-12{,}045, -12{,}015]$, longitud $\in [-77{,}045, -77{,}015]$). Si las coordenadas se ubican fuera de dicho rango, la transacción se aborta con una excepción de dominio, garantizando la integridad geográfica de las denuncias almacenadas.

4. Especificación de Interfaces REST (OpenAPI 3.1)

La Tabla 2 sintetiza los contratos de interfaz HTTP expuestos por la Capa de Presentación y Frontera:

Tabla 2: Catálogo de Endpoints REST del Sistema EcoSystem

Método	**Ruta del Recurso**	**Operación de Dominio Asociada**	**Estado HTTP**	**Subsistema**
`POST`	`/api/v1/auth/register`	Creación de cuenta con asignación de rol ciudadano	`201 Created`	Core / Auth
`POST`	`/api/v1/auth/login`	Validación de credenciales y firma de token JWT	`200 OK`	Core / Auth
`GET`	`/api/v1/containers/nearby`	Consulta espacial por radio (`ST_DWithin` + GiST)	`200 OK`	Geoespacial
`POST`	`/api/v1/containers/{id}/report`	Registro de reporte vecinal y evaluación de umbral	`200 OK`	Geoespacial
`GET`	`/api/v1/routes/optimal-daily`	Generación de circuito logístico mediante TSP	`200 OK`	Ruteo TSP
`POST`	`/api/v1/drop-and-go/request`	Generación de lote reciclable y código identificatorio	`201 Created`	Drop & Go
`POST`	`/api/v1/containers/{id}/empty`	Vaciado de contenedor y disparo de `CollectionEvent`	`200 OK`	Drop & Go
`GET`	`/api/v1/admin/coordinators`	Listado de personal operativo (restringido a `ADMIN`)	`200 OK`	Gobernanza
`PUT`	`/api/v1/governance/parameters`	Actualización dinámica de parámetros de negocio	`200 OK`	Gobernanza
`POST`	`/api/v1/complaints/report-litter`	Recepción de denuncia fotográfica y filtro jurisdiccional	`201 Created`	Fiscalización

5. Síntesis Arquitectónica y Extensibilidad

5.1. Evaluación del Diseño Actual

1. **Aislamiento de Capas y Cohesión:** La combinación de una arquitectura cerrada de cuatro capas con particiones verticales por subdominio mantiene una alta cohesión interna en cada

módulo, aislando las entidades de dominio de los detalles de serialización HTTP (Pydantic) y del motor relacional-espacial (SQLAlchemy/PostGIS).

2. **Flexibilidad Algorítmica y Transaccional:** La aplicación combinada de los patrones **Strategy** y **Adapter** dota al motor de ruteo TSP de tolerancia a fallos frente a servicios cartográficos externos, mientras que el patrón **Observer** permite coordinar transacciones entre los módulos de contenedores, paquetes reciclables y alertas sin introducir dependencias cíclicas.

3. **Eficiencia en Consultas Espaciales:** El diseño del esquema físico sobre tipos `Geography` indexados mediante árboles **GiST** resuelve las operaciones de cercanía geográfica en el motor de base de datos en menos de 150 ms, satisfaciendo las restricciones de latencia de la plataforma.

5.2. Puntos de Extensión Arquitectónica

El diseño actual deja establecidos los contratos de interfaz para la incorporación de dos nuevos patrones en la siguiente fase evolutiva del sistema:

- **Patrón Creacional Factory Method:** Extensión del subsistema de reciclaje para instanciar procesadores polimórficos de valorización y cálculo de puntos según la categoría física del material recepcionado en planta (`EN_PLANTA`).

- **Patrón Estructural Decorator:** Extensión de los observadores de notificación para envolver dinámicamente los mensajes de alerta ciudadana a través de múltiples canales de salida (notificaciones en aplicación, Push web y correo electrónico).